МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

Звіт
до лабораторної роботи №3
з дисципліни «PSoC»

Студент 4-го курсу
спеціальності F7 –
«Комп'ютерна інженерія»
Глеба Василь Васильович

м. Ужгород – 2026 р

ЛАБОРАТОРНА РОБОТА №3

Використання матричної клавіатури для введення інформації у інтегрованому середовищі PSoC®Creator™

Мета: Ознайомитися з принципами підключення та використання матричної клавіатури для введення інформації у мікроконтролерних системах на базі PSoC. Набути навичок розробки проєкту в інтегрованому середовищі PSoC® Creator™, налаштування апаратних компонентів та програмної обробки введених даних.

ЗАВДАННЯ ДО ЛАБОРАТОРНОЇ РОБОТИ

1. На початку, коли всі кнопки відпущені горить колір White. Запрограмувати кнопки так, щоб при натиску на кнопку світлодіод змінював колір за певним алгоритмом (табл. 1). Після натиску, коли кнопка відпущена світлодіод вимикається (Black).

Таблиця 1 - Кольори кнопок

Red	1, 7
Green	2, 8
Blue	3, 9
Yellow	4, *
Purple	5, 0
Cyan	6, #

2. Реалізувати даний проєкт переписавши код програми під використання конструкції Swich(case).
3. Встановити пароль (довжиною > 2 символів). Якщо користувач ввів пароль вірно в консоль виводиться повідомлення «Access allowed», в іншому випадку «Access denied».

Виконання:

1. На початку, коли всі кнопки відпущені горить колір White. Запрограмувати кнопки так, щоб при натиску на кнопку світлодіод змінював колір за певним алгоритмом (табл. 1). Після натиску, коли кнопка відпущена світлодіод вимикається (Black).

Таблиця 1 - Кольори кнопок

Red	1, 7
Green	2, 8
Blue	3, 9
Yellow	4, *
Purple	5, 0
Cyan	6, #

```
/* Function to set LED color based on button */
static void setLEDColor(uint8_t button_value)
{
    switch(button_value)
    {
        case 1:  // Red
        case 7:
            LED_R_Write(0);
            LED_G_Write(1);
            LED_B_Write(1);
            break;

        case 2:  // Green
        case 8:
            LED_R_Write(1);
            LED_G_Write(0);
            LED_B_Write(1);
            break;

        case 3:  // Blue
        case 9:
            LED_R_Write(1);
            LED_G_Write(1);
            LED_B_Write(0);
            break;

        case 4:  // Yellow (Red + Green)
        case 10: // *
            LED_R_Write(0);
            LED_G_Write(0);
            LED_B_Write(1);
            break;

        case 5:  // Purple (Red + Blue)
        case 0:
            LED_R_Write(0);
            LED_G_Write(1);
            LED_B_Write(0);
            break;

        case 6:  // Cyan (Green + Blue)
        case 11: // #
            LED_R_Write(1);
            LED_G_Write(0);
            LED_B_Write(0);
            break;

        default:
            // Turn off LED (Black)
            LED_R_Write(1);
            LED_G_Write(1);
            LED_B_Write(1);
            break;
    }
}
```

2. Реалізувати даний проєкт переписавши код програми під використання конструкції Swich(case).

```c
/* Function to get button name */
static void printButtonName(uint8_t button_value)
{
    switch(button_value)
    {
        case 0:
            SW_Tx_UART_PutString("Button 0 pressed");
            break;
        case 1:
            SW_Tx_UART_PutString("Button 1 pressed");
            break;
        case 2:
            SW_Tx_UART_PutString("Button 2 pressed");
            break;
        case 3:
            SW_Tx_UART_PutString("Button 3 pressed");
            break;
        case 4:
            SW_Tx_UART_PutString("Button 4 pressed");
            break;
        case 5:
            SW_Tx_UART_PutString("Button 5 pressed");
            break;
        case 6:
            SW_Tx_UART_PutString("Button 6 pressed");
            break;
        case 7:
            SW_Tx_UART_PutString("Button 7 pressed");
            break;
        case 8:
            SW_Tx_UART_PutString("Button 8 pressed");
            break;
        case 9:
            SW_Tx_UART_PutString("Button 9 pressed");
            break;
        case 10:
            SW_Tx_UART_PutString("Button * pressed");
            break;
        case 11:
            SW_Tx_UART_PutString("Button # pressed");
            break;
        default:
            break;
    }
    SW_Tx_UART_PutCRLF();
}
```

3. Встановити пароль (довжиною > 2 символів). Якщо користувач ввів пароль вірно в консоль виводиться повідомлення «Access allowed», в іншому випадку «Access denied».

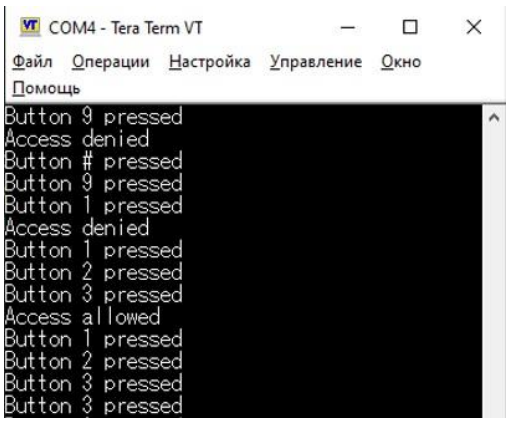


Рисунок 1 – tera term

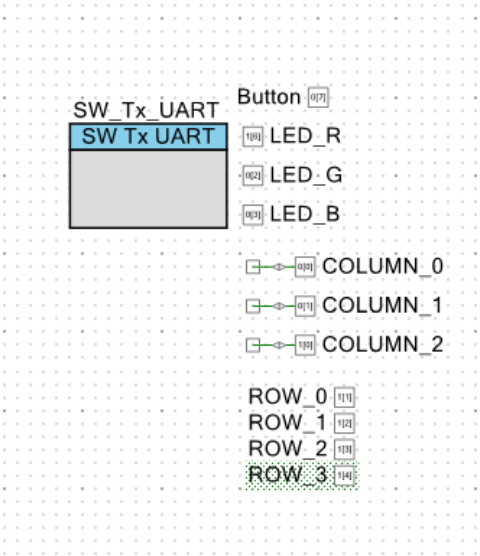
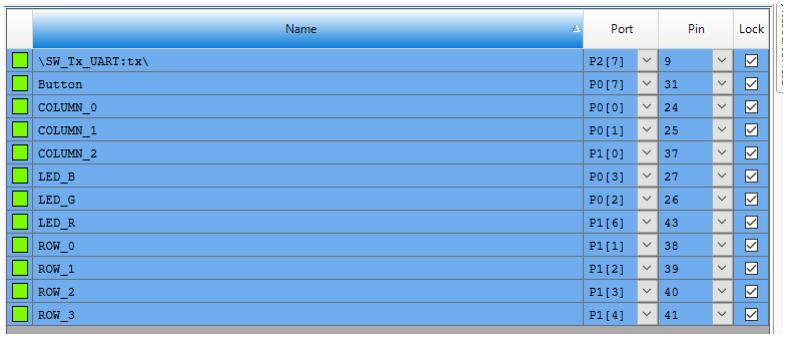


Рисунок 2 - TopDisign.cydsn



Name	Port	Pin	Lock
\SW_Tx_UART:tx\	P2[7]	9	☑
Button	P0[7]	31	☑
COLUMN_0	P0[0]	24	☑
COLUMN_1	P0[1]	25	☑
COLUMN_2	P1[0]	37	☑
LED_B	P0[3]	27	☑
LED_G	P0[2]	26	☑
LED_R	P1[6]	43	☑
ROW_0	P1[1]	38	☑
ROW_1	P1[2]	39	☑
ROW_2	P1[3]	40	☑
ROW_3	P1[4]	41	☑

Рисунок 3 – LWO2.cydwr

Висновок. Ознайомився з принципами підключення та використання матричної клавіатури для введення інформації у мікроконтролерних системах на базі PSoC. Набув навичок розробки проєкту в інтегрованому середовищі PSoC® Creator™, налаштування апаратних компонентів та програмної обробки введених даних.

Лістинг коду:

```
#include "project.h"

/* arrays of pointers */
/* function of drive mode configuration */
static void (*COLUMN_x_SetDriveMode[3])(uint8_t mode) = {
    COLUMN_0_SetDriveMode,
    COLUMN_1_SetDriveMode,
    COLUMN_2_SetDriveMode
};
/* column write function */
static void (*COLUMN_x_Write[3])(uint8_t value) = {
    COLUMN_0_Write,
    COLUMN_1_Write,
    COLUMN_2_Write
};
```

```
/* read row function */
static uint8 (*ROW_x_Read[4])() = {
    ROW_0_Read,
    ROW_1_Read,
    ROW_2_Read,
    ROW_3_Read
};

/* [ROW][COLUMN] */
static uint8_t keys[4][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
    {10, 0, 11},
};

/* Password settings */
#define PASSWORD_LENGTH 3
static const uint8_t PASSWORD[PASSWORD_LENGTH] = {1, 2, 3}; // Встановіть
свій пароль тут
static uint8_t password_buffer[PASSWORD_LENGTH];
static uint8_t password_index = 0;
static uint8_t password_locked = 1; // 1 = заблоковано, 0 = розблоковано

/* matrix initialization function */
static void initMatrix()
{
    for(int column_index = 0; column_index < 3; column_index++)
    {
         COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

/* keys matrix read function */
static void readMatrix()
{
    /* define the length of a row and column */
    uint8_t row_counter = sizeof(ROW_x_Read)/sizeof(ROW_x_Read[0]);
    uint8_t column_counter =
sizeof(COLUMN_x_Write)/sizeof(COLUMN_x_Write[0]);
    /* column: iterate the columns */
    for(int column_index = 0; column_index < column_counter; column_index++)
{
        COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_STRONG);
        COLUMN_x_Write[column_index](0);
        /* row: interate throught the rows */
        for(int row_index = 0; row_index < row_counter; row_index++)
        {
            keys[row_index][column_index] = ROW_x_Read[row_index]();
        }
        /* disable the column */
        COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

/* Function to check password */
static void checkPassword()
{
    uint8_t correct = 1;
    for(int i = 0; i < PASSWORD_LENGTH; i++)
    {
        if(password_buffer[i] != PASSWORD[i])
        {
            correct = 0;
```

```
            break;
        }
    }

    if(correct)
    {
        SW_Tx_UART_PutString("Access allowed");
        SW_Tx_UART_PutCRLF();
        password_locked = 0;
        // Білий світлодіод при розблокуванні
        LED_R_Write(0);
        LED_G_Write(0);
        LED_B_Write(0);
        CyDelay(1000);
    }
    else
    {
        SW_Tx_UART_PutString("Access denied");
        SW_Tx_UART_PutCRLF();
        password_locked = 1;
    }

    password_index = 0;
}

/* Function to set LED color based on button */
static void setLEDColor(uint8_t button_value)
{
    switch(button_value)
    {
        case 1:  // Red
        case 7:
            LED_R_Write(0);
            LED_G_Write(1);
            LED_B_Write(1);
            break;

        case 2:  // Green
        case 8:
            LED_R_Write(1);
            LED_G_Write(0);
            LED_B_Write(1);
            break;

        case 3:  // Blue
        case 9:
            LED_R_Write(1);
            LED_G_Write(1);
            LED_B_Write(0);
            break;

        case 4:  // Yellow (Red + Green)
        case 10: // *
            LED_R_Write(0);
            LED_G_Write(0);
            LED_B_Write(1);
            break;

        case 5:  // Purple (Red + Blue)
        case 0:
            LED_R_Write(0);
            LED_G_Write(1);
            LED_B_Write(0);
            break;
```

```
        case 6:  // Cyan (Green + Blue)
        case 11: // #
            LED_R_Write(1);
            LED_G_Write(0);
            LED_B_Write(0);
            break;

        default:
            // Turn off LED (Black)
            LED_R_Write(1);
            LED_G_Write(1);
            LED_B_Write(1);
            break;
    }
}

/* Function to get button name */
static void printButtonName(uint8_t button_value)
{
    switch(button_value)
    {
        case 0:
            SW_Tx_UART_PutString("Button 0 pressed");
            break;
        case 1:
            SW_Tx_UART_PutString("Button 1 pressed");
            break;
        case 2:
            SW_Tx_UART_PutString("Button 2 pressed");
            break;
        case 3:
            SW_Tx_UART_PutString("Button 3 pressed");
            break;
        case 4:
            SW_Tx_UART_PutString("Button 4 pressed");
            break;
        case 5:
            SW_Tx_UART_PutString("Button 5 pressed");
            break;
        case 6:
            SW_Tx_UART_PutString("Button 6 pressed");
            break;
        case 7:
            SW_Tx_UART_PutString("Button 7 pressed");
            break;
        case 8:
            SW_Tx_UART_PutString("Button 8 pressed");
            break;
        case 9:
            SW_Tx_UART_PutString("Button 9 pressed");
            break;
        case 10:
            SW_Tx_UART_PutString("Button * pressed");
            break;
        case 11:
            SW_Tx_UART_PutString("Button # pressed");
            break;
        default:
            break;
    }
    SW_Tx_UART_PutCRLF();
}
```

```c
int main(void)
{
    CyGlobalIntEnable; /* Enable global interrupts. */

    SW_Tx_UART_Start();
    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Software Transmit UART");
    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Enter password to unlock");
    SW_Tx_UART_PutCRLF();

    initMatrix();

    uint8_t last_state = 255; // Ніяка кнопка не натиснута
    uint8_t button_pressed = 0;

    // Початковий стан - білий світлодіод (система заблокована)
    LED_R_Write(0);
    LED_G_Write(0);
    LED_B_Write(0);

    for(;;)
    {
        readMatrix();
        button_pressed = 0;

        // Сканування всіх кнопок
        for(int row = 0; row < 4; row++)
        {
            for(int col = 0; col < 3; col++)
            {
                if(keys[row][col] == 0) // Кнопка натиснута
                {
                    uint8_t button_value = (row == 3 && col == 1) ? 0 :
                                           (row == 3 && col == 0) ? 10 :
                                           (row == 3 && col == 2) ? 11 :
                                           (row * 3 + col + 1);

                    if(last_state != button_value)
                    {
                        last_state = button_value;
                        printButtonName(button_value);

                        // Якщо система заблокована, збираємо пароль
                        if(password_locked)
                        {
                            password_buffer[password_index] = button_value;
                            password_index++;

                            if(password_index >= PASSWORD_LENGTH)
                            {
                                checkPassword();
                            }
                        }
                        else
                        {
                            // Система розблокована, змінюємо колір
                            setLEDColor(button_value);
                        }
                    }
                    button_pressed = 1;
                    break;
                }
            }
```

```
            if(button_pressed) break;
        }

        // Якщо жодна кнопка не натиснута
        if(!button_pressed)
        {
            if(last_state != 255)
            {
                last_state = 255;

                if(password_locked)
                {
                    // Білий світлодіод, коли система заблокована і кнопка
відпущена
                    LED_R_Write(0);
                    LED_G_Write(0);
                    LED_B_Write(0);
                }
                else
                {
                    // Вимкнути світлодіод (Black), коли система розблокована
і кнопка відпущена
                    LED_R_Write(1);
                    LED_G_Write(1);
                    LED_B_Write(1);
                }
            }
        }

        CyDelay(50); // Debounce delay
    }
}
```